# Regression — Key Concepts

Lesson 3 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

16 October 2026 · a one-page index

## Contents

The model and its cost . . . . . 1
Solving it exactly . . . . . 2
Solving it iteratively . . . . . 2
Bending the line . . . . . 2
Regularisation . . . . . 3
Reading the answer . . . . . 3

An index, not a summary. Each entry says what a term means and where the handout treats it properly. Use this to find your way back into the text, or to check before an exercise that there is no word here you could not define.

### The model and its cost

**Linear model.** Each feature contributes a fixed amount per unit and the contributions add up. A strong claim, often false, and worth knowing you have made. → § 2.1

**Mean squared error (MSE).** The average squared gap between prediction and truth. Squaring makes every error count positively and makes one large mistake cost far more than several small ones. → § 2.2

**Why squared rather than absolute.** Differentiable everywhere, it punishes large errors harder, and under Gaussian noise minimising it *is* maximum likelihood — which is the reason that makes it more than a convention. → § 2.2

**Sensitivity to outliers.** A consequence of the same squaring: one absurd value contributes its error squared and drags the whole fit towards itself. → § 2.2

**Baseline, for a regression.** Always predict the mean of the target. The regression counterpart of the majority-class classifier, and what $R^2$ measures a model against. → § 2.4

**Coefficient of determination ($R^2$).** One minus the model's squared error over the squared error of always predicting the mean: 1 is perfect, 0 is the mean, and negative is worse than the mean. Divided by the variance of the test set used, so it does not compare models measured on different data. → § 2.4

## Solving it exactly

**Design matrix.** The training data as a matrix, with a column of ones in front so the intercept is just another coefficient. → § 3.1

**Normal equation.** $\theta = (X^\top X)^{-1} X^\top y$: the exact minimiser, obtained by setting the gradient to zero. → § 3.1

**Orthogonal projection.** What least squares does geometrically: the fitted values are the shadow of the target on the space the columns can reach, and the residual is perpendicular to it. → § 3.1

**Convexity.** $X^\top X$ is positive semi-definite, so the cost is a bowl and any stationary point is the global minimum. There are no local minima to get stuck in. → § 3.1

**Singular and near-singular matrices.** Exactly redundant columns make the inverse impossible; nearly redundant ones make it enormous and unstable, which is worse because nothing fails visibly. → § 3.3

## Solving it iteratively

**Gradient descent.** Start somewhere, feel which way the ground slopes, step against it, repeat. → § 4.1

**The gradient as a weighted vote.** Each example pulls a coefficient in proportion to two things: how wrong the prediction was, *and* how large that feature was for that example. → § 4.1

**Learning rate $\alpha$.** The length of the stride — the one parameter gradient descent cannot choose for itself. Too small crawls, too large climbs the opposite wall and diverges. → § 4.3

**The contraction factor $1 - \alpha c$.** What one step multiplies the distance to the minimum by, on a parabola of curvature $c$. Every regime of the learning rate is this one number being inside, on, or outside $(-1, 1)$. → § 4.3

**The stability threshold $\alpha < 2/c$.** Where that factor leaves $(-1, 1)$, and therefore where descent stops converging. With several features the largest curvature binds. → § 4.3

**Condition number.** The ratio of the steepest curvature to the shallowest: how stretched the valley is, and near enough what sets the number of iterations you will need. The curvatures are the eigenvalues of $X^\top X$, so it equals `np.linalg.cond(X)` squared, not `np.linalg.cond(X)` itself. → § 4.4

**Why iterations grow with it.** Spending the largest safe rate on the steep direction leaves the shallow one contracting by $1 - 1/\kappa$ per step, so the steps needed grow in proportion to $\kappa$. → § 4.4

## Bending the line

**Linear in the coefficients.** "Linear" constrains how the coefficients enter, not what the inputs are. Hand the model $x^2$ as a column and the same normal equation fits a curve. → § 5.1

**Root mean squared error (RMSE).** The square root of the mean squared error, which puts the number back into the units of the target. → § 5.2

**Training error against test error.** Training error falls with every degree added, monotonically, without ever warning you. Test error turns around, and where it turns is the model you want. → § 5.2

**Why flexible models misbehave.** Not that they must wiggle — that the arrangement least squares finds involves large coefficients working in opposition, cancelling exactly where a training point sits and swinging freely between. → § 5.3

## Regularisation

**Ridge ($L_2$).** Add a charge proportional to the sum of squared coefficients. Shrinks everything towards zero without ever reaching it. → § 6.1

**Lasso ($L_1$).** Add a charge proportional to the sum of absolute values. Reaches exactly zero at a finite penalty, which makes it a feature selector. → § 6.1

**Penalty strength $\lambda$.** The exchange rate between fitting the data and keeping coefficients small. Note that scikit-learn spells this parameter `alpha`, which is the learning rate in these pages. → § 6.1

**Why scaling is mandatory here.** The penalty is a sum over coefficients, so the same feature in metres and in kilometres attracts penalties differing by a thousand. A matter of correctness, not speed. → § 6.1

**Ridge's closed form.** $\theta = (X^\top X + \tilde{\lambda} I)^{-1} X^\top y$. Adding to the diagonal lifts every eigenvalue, so the inverse always exists — Ridge has an answer where least squares has none. → § 6.2

**Why Lasso reaches zero.** Its constraint region is a diamond whose corners lie on the axes, and a corner is where a coefficient is exactly zero. Corners protrude, so they get touched first. → § 6.3

## Reading the answer

**What a coefficient means.** The change in the prediction for a one-unit change in that feature, *with every other feature held fixed* — a clause that does enormous work and is almost always glossed over. → § 7.1

**Correlated features.** A coefficient is only as trustworthy as the independence of its feature: the ones sharing information with others are the ones recovered worst. → § 7.2

**Collinearity.** Two columns carrying nearly the same information. Predictions stay fine while the coefficients swing wildly between splits, because the fit has almost no basis for dividing the effect. → § 7.3

**Association, not causation.** A coefficient describes a pattern in the training data. Nothing in it tells you what would happen if you intervened. → § 7.4